

Estate Manager Web App — Product Strategy, Architecture & Phased Execution Blueprint

Author: Executive Product Manager

Scope: Prototype & MVP 1 (Full Production-Grade Specification)

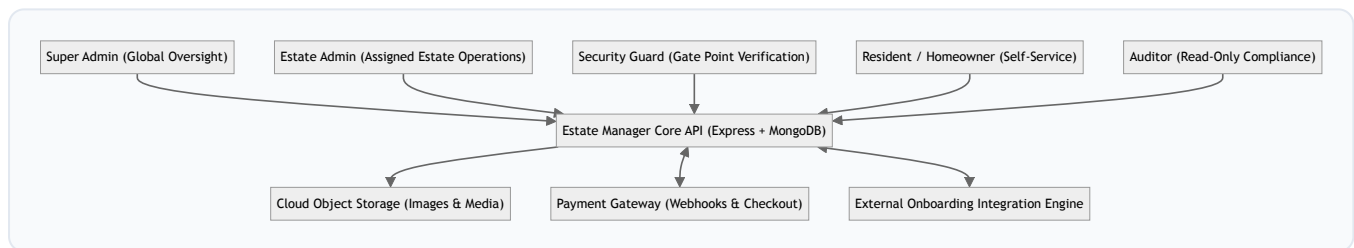
Platform: Modern Multi-Tenant Web Application & Progressive Web App (PWA)

Reference Document: ESTATE MANAGER WEB APP brd.pdf (40-Page Requirements Specification)

1. Executive Product Summary & Core Vision

The **Estate Manager Web App** is an enterprise-grade, multi-estate property management and gate security ecosystem. It bridges the operational friction between **Residents/Homeowners**, **Facility Management (FM) Teams**, **Security Personnel**, and **Executive Administrators** across multiple residential estates in a single, unified platform.

Unlike legacy software, Estate Manager is designed from day one with **multi-estate scaling**, **zero-trust security**, **consumer-grade user experience**, and **offline-resilient PWA capabilities**.



Supported Personas & Capabilities

- Resident / Homeowner:** Multi-property management, instant guest/service access codes, invoice tracking & one-tap payment, 2-way facility maintenance requests, facility reservation, emergency SOS broadcasting.
- Security Guard:** Sub-5-second gate verification, numeric/QR code validation, mandatory visitor & vehicle photo capture, zero access to resident financials or private chats.
- Estate Admin (FM Team):** Estate-scoped dashboard, property & resident directories, bulk/single invoice generation, ticket management with SLA tracking, estate-wide broadcasting, gate activity monitoring.
- Super Admin:** Global estate oversight, administrative permission delegation, financial reconciliation, audit logs, and the exclusive authority to restrict resident access-code generation for arrears without locking the resident out of their account.
- Auditor:** Immutable, read-only oversight across financials, access histories, verification logs, and administrative actions.

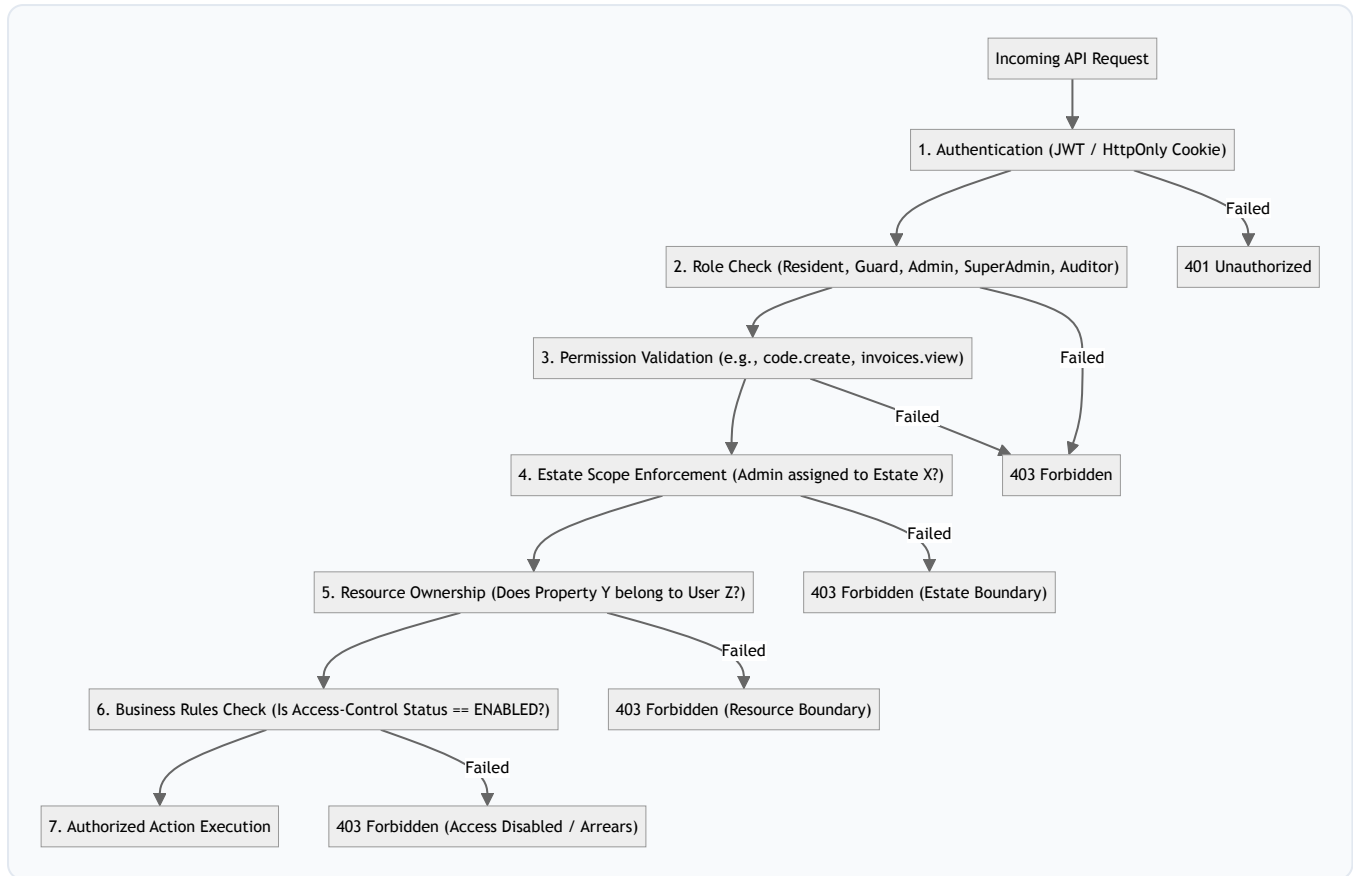
2. Competitive Teardown: Dethroning 14-Year Legacy Incumbents

Incumbents in the residential and property management sector (e.g., legacy systems built 10–14+ years ago) suffer from systemic architectural rot and obsolete UX patterns. To win their market share, Estate Manager directly targets their structural vulnerabilities:

Operational Area	Legacy 14-Year Incumbents	Estate Manager Web App (Our Advantage)
Gate Verification Speed	Clunky forms, 45–60s verification time, slow camera uploads, causing traffic gridlocks at gates.	Sub-5-second verification. Optimistic UI, instant camera viewfinder, background parallel photo upload.
Device & App Distribution	Heavy native apps that require constant app store updates or non-responsive desktop sites.	Zero-install PWA. Installs directly from browser onto iOS, Android, and desktop tablets with native push notifications.
Debt Enforcement vs UX	Brutal Account Lockout: Users are blocked from logging in, meaning they cannot view invoices, verify arrears, or make online payments.	Surgical Access-Code Restriction (Rules 11-13): Super Admin restricts code generation only; billing, complaints, and instant payment portals remain 100% active.
Property Relationships	Rigid 1 User = 1 Unit database models. Landlords with multiple houses need multiple accounts.	Decoupled Relational Model (User $\rightarrow$ UserProperty $\rightarrow$ Property $\rightarrow$ Estate): Unified resident dashboard spanning multiple estates and units.
Security Architecture	Prone to Broken Object Level Authorization (BOLA/IDOR); changing URL query parameters exposes other estates' data.	7-Layer Defense-in-Depth Authorization Pipeline: Every request enforces role, permission, estate boundary, and resource ownership at the backend.
Network Resilience	Drops dead when 3G/4G fails at the estate gate or basement checkpoints.	Service Worker Offline Cache & Queue: Guards can validate cached credentials and queue image uploads reliably.

3. Core Architecture & 7-Layer Security Model

The system enforces Section 63 of the BRD through a mandatory 7-layer verification pipeline executed on every single protected backend route:



Security Engineering Specifications

- **Short-Lived JWT Access Tokens:** 30-minute expiry (`JWT_EXPIRES_IN=30m`), minimal payload claims (no PII inside tokens).
- **HttpOnly, Secure, SameSite Cookies:** Immune to XSS token theft via browser JavaScript; CSRF token validation on mutating endpoints.
- **Strict Content Security Policy (CSP):** Configured in `client/index.html` and enforced via HTTP headers (`default-src 'self'` , whitelisted storage & payment domains).
- **No Direct S3/Blob in DB:** Storage abstraction (`storageService`) handles visitor & vehicle images via presigned URLs, MIME validation, file size limits, and sanitization. Filenames from clients are discarded and replaced with cryptographic UUIDs.
- **Cryptographic Access Codes:** Unpredictable, hashed storage (`codeHash`), scoped to property/estate, strictly time-bound.
- **Immutable Audit Logging:** All sensitive administrative mutations (billing, permission changes, access restrictions, manual approvals) record actor ID, previous/new state, client IP, and user-agent.

4. User Experience (UX) & Design Strategy

Resident Interface (Consumer Delight)

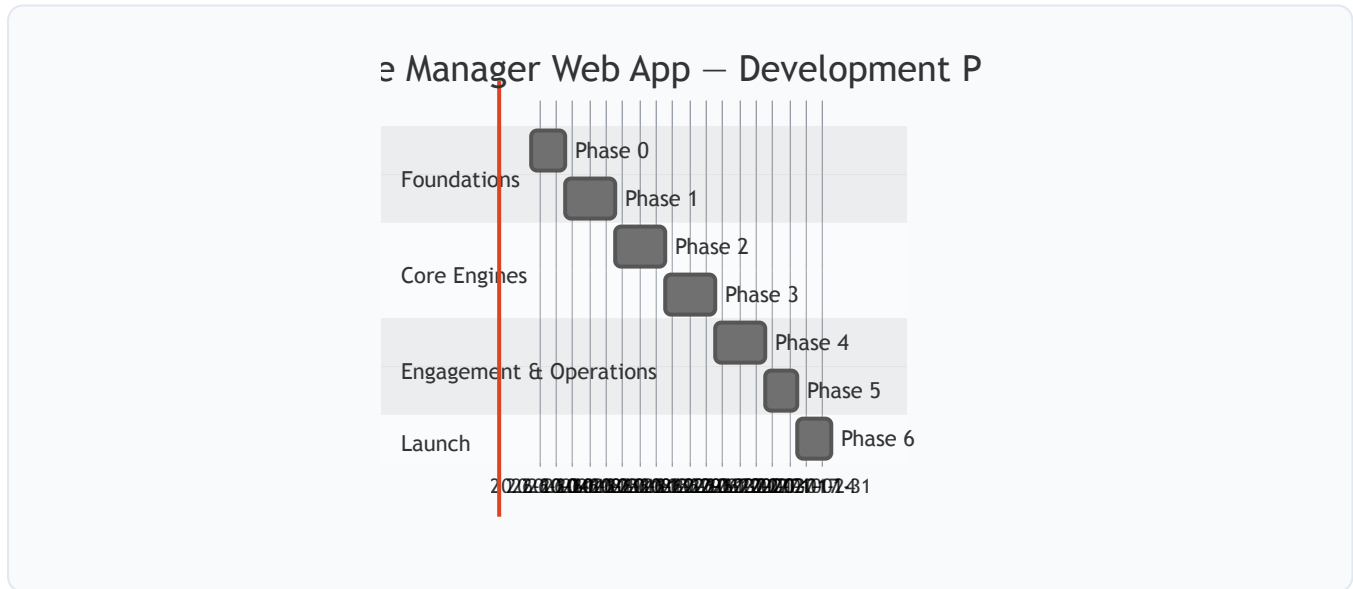
- **Speed & Clarity:** Inspired by modern fintech apps (Stripe, Linear, Revolut). Dashboard loads within 300ms with cached TanStack Query state.

- **Unified Property Switcher:** Clear dropdown/pill switcher to toggle between "Court A - A223 (Mansfield)" and "Plot 12 (Fairfield)".
- **One-Tap Access Code Generation:** Quick presets (Visitor, Delivery, Artisan, Cab) with pre-filled expiry rules and one-touch "Share via WhatsApp / Copy" action.
- **Transparent Debt Status:** If code generation is restricted, a polite, non-punitive banner explains the arrears with a direct "Pay Outstanding Balance" button that immediately re-enables their codes upon webhook verification.

Security Guard Interface (High-Velocity Gate Station)

- **Ergonomic Gate Workflow:** Designed for outdoor sunlight and night shifts (high-contrast mode, massive numeric keys, oversized confirmation buttons).
- **Sub-5-Second Throughput:**
 1. Guard types or scans the 6-digit access code.
 2. Instant verification status with visitor name and destination property.
 3. One-tap integrated camera capture (Visitor Photo mandatory; Vehicle Photo mandatory if flagged).
 4. Instant "Approve & Log Entry" click.
- **Strict Privacy Shield:** Guard UI reveals **zero** financial data, invoices, resident complaints, or unrelated estate details.

5. End-to-End Phased Development Breakdown



Phase 0: Design System, Architecture & Security Foundation

Goal: Lay the unbreakable architectural blueprint, design tokens, and defensive security baselines

before writing business logic.

1. Engineering Deliverables

- Initialize repository structure following Section 56 (`client/` , `server/` , `shared/`).
- Configure Vite + React + Tailwind CSS with dark/light themes and high-contrast gate mode.
- Establish backend Express structure with Helmet, CORS allowlisting, rate limiting (`express-rate-limit`), and centralized error-handling middleware.
- Setup Mongoose connection with connection pooling, MongoDB query sanitization, and global schema plugins (timestamps, soft-delete, audit hooks).
- Implement CSP meta/headers and sanitize input with Zod validation schemas shared between frontend and backend.

2. Design & UX Deliverables

- Create Figma design tokens: Typography, color hierarchy (Emerald for active/approved, Amber for pending, Rose for overdue/rejected, Indigo for primary actions).
- Design responsive layouts:
 - Resident PWA layout (bottom navigation bar for mobile, clean sidebar for desktop).
 - Guard Gate interface (optimized for single-handed tablet or phone operation at security booths).
 - Admin & Super Admin command dashboard (collapsible multi-tier sidebar, high-density data tables).

3. Security & Compliance

- Setup automated test harness (Vitest / Supertest) configured to execute security test cases.
- Implement short-lived JWT + HttpOnly SameSite cookie authentication flow.

Phase 1: Multi-Tenancy, Identity & External Onboarding Sync

Goal: Build the multi-estate property data model and the idempotent ingestion engine for external onboarding.

1. Engineering Deliverables

- **Data Models:**
 - `User` : Full profile, role, password hash (Argon2), `accessControlStatus` (`ENABLED` / `DISABLED`), restriction metadata.
 - `Estate` : Name, code, address, gate configuration (require visitor image, require vehicle image).
 - `Property` : Type (Apartment, Terrace, Land, Penthouse, etc.), Court, Block, Floor, Apartment number, display identifier.
 - `UserProperty` : Junction table linking `userId` , `propertyId` , `estateId` , and relationship type (Owner, Tenant).

- `Dependant` : Associated with resident, protected with permission-gated visibility.
- `EstateAdminAssignment` : Explicit junction mapping admins to specific estates.
- **External Onboarding Integration (`/api/integrations/onboarding`):**
 - REST endpoints supporting `PUSH` , `GET` , `PULL` , and `SYNC` .
 - Idempotent payload handling: Uses external onboarding ID to ensure duplicate sync events never create duplicate users or properties.
 - Backend-only integration: Direct database access from frontend is strictly blocked.
- **RBAC & Scoping Middleware:**
 - Role checking (`Resident` , `Guard` , `Admin` , `SuperAdmin` , `Auditor`).
 - Estate-scoping middleware that verifies `req.user.assignedEstates` matches the requested resource.

2. Design & UX Deliverables

- Resident multi-property switcher component with persistent state in session/local storage.
- Admin resident management table: Search, multi-column sort, estate filtering, expandable row for authorized dependant count (with granular drawer for sensitive details).
- Self-service resident profile view with read-only locked fields for property assignments.

Phase 2: Gate Security & Access Control Engine (The Competitive Moat)

Goal: Deliver the sub-5-second gate verification experience and the surgical debt-enforcement access code control.

1. Engineering Deliverables

- **Access Code Generation Engine (`/api/codes`):**
 - Cryptographically random 6-digit numeric or alphanumeric codes.
 - Codes hashed in database (`codeHash`); raw code only returned upon creation.
 - Parameters: `residentId` , `propertyId` , `estateId` , `type` , `expiresAt` , `validationMethods` (`visitor_image` , `vehicle_image`), `maxUses` .
 - **Rule 8 Enforcement:** Resident ID is strictly purged and rejected from all validation schemas.
- **Super Admin Code Restriction Engine (Rules 11, 12, 13):**
 - Dedicated endpoint: `POST /api/users/:id/restrict-access-code` .
 - Only `SuperAdmin` can execute. Sets `accessControlStatus = DISABLED` , records reason, timestamp, and actor ID.
 - Automatically invalidates existing active codes for that resident.
 - Triggers push/in-app notification to resident.
 - Generates immutable `AuditLog` .
 - Backend guards `POST /api/codes` : Returns `403 ACCESS_CONTROL_DISABLED` if restricted.

- **Guard Verification Engine (/api/verifications):**

- Real-time code validation against estate ID, property ID, expiry timestamp, and resident access status.
- Image upload pipeline (`storageService`): Guard captures photo via HTML5 Camera API / `MediaDevices`; backend validates image headers, resizes, and saves to object storage.
- Enforces mandatory visitor image (`400 REQUIRED_VISITOR_IMAGE`) and vehicle image if configured.
- Creates immutable `VerificationLog` with guard ID, IP, device ID, and timestamp.
- Triggers instant real-time gate notification to the resident.

2. Design & UX Deliverables

- **Guard Station PWA:**

- Large keypad + camera viewfinder with automatic auto-focus and preview/retake toggles.
- Instant tactile feedback (audio tone + vibration for valid/invalid code).

- **Resident Code Center:**

- Active code card with copy-to-clipboard, countdown timer to expiry, and QR code modal.
- If restricted: Warning card explaining the arrears with contact/payment CTA without disabling navigation.

Phase 3: Invoicing, Billing & Bulletproof Payment Verification

Goal: Enable multi-tier billing and zero-trust, server-side verified payment flows.

1. Engineering Deliverables

- **Invoice Engine (/api/invoices):**

- Data schema: `invoiceNumber`, `userId`, `propertyId`, `estateId`, `amount`, `amountPaid`, `balance`, `dueDate`, `status` (`DRAFT`, `PENDING`, `PARTIALLY_PAID`, `PAID`, `OVERDUE`, `CANCELLED`).
- Single invoice creation & bulk billing wizard (target an entire estate or court).

- **Payment Processing (/api/payments):**

- Integration with payment gateway checkout.
- Webhook listener (`POST /api/payments/webhook`):
 - Validates gateway cryptographic signature (HMAC-SHA256).
 - Idempotency guard: Replay of the same webhook reference returns `200 OK` without duplicating ledger entries.
 - Server-side amount verification: Compares webhook amount against invoice balance.
 - Atomically updates invoice balance and status to `PAID` or `PARTIALLY_PAID`.
 - Generates immutable payment record and resident notification.
- **Rule 15 & Rule 16:** Frontend is completely incapable of marking an invoice as paid.

2. Design & UX Deliverables

- Resident billing portal: Invoice breakdown, partial vs full payment options, downloadable PDF receipts.
- Admin finance dashboard: Total outstanding arrears, collection rates by estate, overdue aging buckets.
- Bulk billing modal with live preview of affected residents and total projected revenue before confirmation.

Phase 4: Facility Operations, Two-Way Comms, SOS & Community

Goal: Build the day-to-day engagement layer for maintenance, amenities, emergencies, and estate communications.

1. Engineering Deliverables

- **Complaint Management (/api/complaints):**
 - Tickets with categories (Plumbing, Electrical, Security, Noise), media attachments, and priority.
 - Status pipeline: OPEN $\rightarrow$ ASSIGNED $\rightarrow$ IN_TREATMENT $\rightarrow$ RESOLVED $\rightarrow$ CLOSED .
 - Real-time two-way messaging between resident and assigned FM staff.
- **Estate Message Board (/api/messages):**
 - Broadcast engine scoped to all estates, single estate, or specific property types.
 - Categories: GENERAL , ANNOUNCEMENT , INVOICE_REMINDER , MAINTENANCE , SECURITY .
- **Facility Booking (/api/facilities & /api/bookings):**
 - Facilities definition (operating hours, capacity, booking rules).
 - Overlap prevention algorithm: Prevents double-booking of slots using MongoDB atomic query conditions.
- **Emergency SOS (/api/sos):**
 - One-tap SOS trigger sending real-time socket events and high-priority push notifications to FM staff and gate guards.
 - Stores location, property unit, and timestamp. Admin dashboard displays pulsing alert banners.
- **Community Feed (/api/community):**
 - Intra-estate discussion feed strictly isolated by estate boundaries.

2. Design & UX Deliverables

- Kanban/Card complaint management board for Estate Admins with color-coded status badges.
- Facility calendar with interactive time-slot picker (green for open, grey for reserved).
- Prominent, guarded SOS button with a 3-second cancel countdown to prevent accidental triggers.

Phase 5: PWA Hardening, Offline Guard Resilience & Comprehensive Audit Suite

Goal: Optimize for mobile performance, offline gate operation, and enterprise regulatory compliance.

1. Engineering Deliverables

- **PWA Architecture:**
 - Manifest configuration (standalone display, theme colors, icons).
 - Service worker caching strategy:
 - App shell & static assets: Cache-first.
 - Resident API queries: Network-first with TanStack Query background hydration.
 - Guard verification queue: Background sync to store offline photo captures and sync when signal returns.
 - Web Push notification integration for immediate gate entry and payment alerts.
- **Comprehensive Audit Log Explorer (/api/audit-logs):**
 - Filter by actor, role, estate, date, action, and target resource.
 - Dedicated read-only Auditor portal satisfying Section 7 & Section 50 of BRD.
- **Security Penetration & Automated Validation:**
 - Verification of all 61 critical security test cases (BOLA, IDOR, brute-force rate limits, MIME validation).

2. Design & UX Deliverables

- Seamless "Add to Home Screen" prompt for residents and guards with customized installation onboarding.
- Auditor dashboard featuring exportable compliance reports (CSV/PDF) and visual audit timelines.

Phase 6: Legacy Migration, Field Pilot & Go-To-Market

Goal: Seamlessly transition estates away from 14-year legacy systems without operational downtime.

1. Engineering Deliverables

- **Legacy ETL Migration Pipeline:**
 - CSV/JSON ingestion script to map legacy database schemas (tenants, property numbers, historic payment balances) into Estate Manager models.
 - Pre-migration data sanitization (deduplicating phone numbers, normalizing court/unit naming conventions).
- **Telemetry & Health Monitoring:**
 - Real-time error reporting (Sentry), API latency tracking, gate verification latency metrics.

2. Operational & Rollout Deliverables

- Conduct a 14-day parallel pilot at a selected residential estate.
 - Security guard field training (target: under 3 minutes to train a new guard on the 3-tap workflow).
 - Monitor key performance indicators (KPIs): Gate verification turnaround time (< 5s), zero failed webhooks, resident app adoption rate (> 85% within 30 days).
-

6. Mandatory Business Rules & Test Verification Matrix

Every rule from Section 62 of the BRD is mapped to its exact architectural enforcement point:

Rule #	Business Rule Specification	Backend Enforcement Mechanism	Critical Security Test Assertion
Rule 1	A resident may have multiple properties.	<code>UserProperty</code> junction model.	Test that 1 <code>User</code> can bind to <code>\$N\$Property</code> documents across multiple estates.
Rule 2	Every property belongs to an estate.	<code>Property.estateId</code> is required and indexed.	Orphan property creation fails validation.
Rule 3	Property relationships independently authorized.	Ownership middleware checks <code>UserProperty</code> record.	Resident cannot query Property B if only authorized on Property A.
Rule 4	Admin can only access assigned estates.	<code>EstateAdminAssignment</code> verification middleware.	Admin of Estate A modifying Estate B returns <code>403 Forbidden</code> .
Rule 5	Super Admin has global administrative access.	Super Admin role bypasses estate-assignment filter.	Super Admin can inspect all estates without assignment record.
Rule 6	Auditor is strictly read-only.	Auditor role is rejected by <code>POST</code> , <code>PUT</code> , <code>PATCH</code> , <code>DELETE</code> .	Auditor executing <code>POST /api/invoices</code> returns <code>403 Forbidden</code> .
Rule 7	Guard access is limited to security verification.	Guard token rejected on <code>/api/invoices</code> , <code>/api/complaints</code> , etc.	Guard requesting resident financial endpoints returns <code>403 Forbidden</code> .
Rule 8	Resident ID is NOT a visitor validation method.	<code>Resident ID</code> omitted from validation enums and UI.	Request with <code>validationMethod: "resident_id"</code> returns <code>400 Bad Request</code> .
Rule 9	Visitor image capture mandatory when configured.	Controller checks <code>req.files['visitorImage']</code> .	Missing visitor image returns <code>400 REQUIRED_VISITOR_IMAGE</code> .
Rule 10	Vehicle image capture mandatory when configured.	Controller checks <code>req.files['vehicleImage']</code> .	Missing vehicle image when required returns <code>400 REQUIRED_VEHICLE_IMAGE</code> .
Rule 11	Super Admin can disable code generation for arrears.	Endpoint <code>/api/users/:id/restrict-access-code</code> .	Only Super Admin can mutate <code>accessControlStatus</code> .
Rule 12	Disabling code generation does NOT suspend account.	Resident token remains valid for dashboard, billing, complaints.	Restricted resident can log in and view invoices successfully.

Rule #	Business Rule Specification	Backend Enforcement Mechanism	Critical Security Test Assertion
Rule 13	Backend must enforce code restriction.	POST /api/codes verifies <code>user.accessControlStatus == ENABLED</code> .	Restricted resident generating code returns <code>403</code> <code>ACCESS_CONTROL_DISABLED</code> .
Rule 14	Sensitive actions must be audited.	Global audit interceptor on mutating admin routes.	Disabling codes, updating invoices, or reallocating creates <code>AuditLog</code> .
Rule 15	Payment success verified server-side.	Webhook crypto verification + gateway query.	Direct client call <code>PATCH /api/invoices/:id/pay</code> returns <code>404</code> or <code>403</code> .
Rule 16	Frontend security never replaces backend auth.	Zero frontend-trust principle; full route validation.	Manually crafted curl requests bypassing UI rules are rejected.

7. Technology Stack & Project Structure Blueprint

Verified Stack

- **Client (PWA):** React 18 / 19 + Vite + Tailwind CSS + Lucide React + React Router v6 + TanStack Query (v5) + React Hook Form + Zod + Vite PWA Plugin.
- **Server:** Node.js + Express.js + Mongoose (MongoDB) + Argon2 / bcrypt + jsonwebtoken + Helmet + CORS + express-rate-limit + Multer (streamed to Cloud Storage).
- **Shared:** Common constants, permissions, validation schemas (Zod).

Codebase Directory Layout

```

estate-manager/
├── client/
│   ├── public/
│   │   ├── favicon.ico
│   │   ├── manifest.json
│   │   └── icons/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── common/      # Buttons, Modals, FormInputs, Badges
│   │   │   ├── guard/      # CameraViewfinder, CodePad, VerificationCard
│   │   │   ├── resident/   # PropertySelector, CodeGenerator, InvoiceCard
│   │   │   └── admin/      # DataTable, MetricWidget, AuditDrawer
│   │   ├── layouts/        # AuthLayout, ResidentLayout, GuardLayout, AdminLayout
│   │   ├── pages/
│   │   │   ├── auth/       # Login, ResetPassword
│   │   │   ├── resident/   # Dashboard, Properties, Bills, AccessCodes, Complaints
│   │   │   ├── guard/      # VerificationStation, ScanHistory
│   │   │   └── admin/      # Overview, Residents, Invoices, Gates, Audits
│   │   ├── hooks/          # useAuth, useActiveProperty, usePWAInstall, useCamera
│   │   ├── services/       # api.js, authService, codeService, paymentService
│   │   ├── context/        # AuthContext, PropertyContext, ToastContext
│   │   ├── utils/          # formatters, validators
│   │   └── routes/         # ProtectedRoute, RoleGuard, AppRoutes
│   └── index.html
├── server/
│   ├── config/             # db.js, env.js, storage.js
│   ├── controllers/        # auth, user, estate, property, code, invoice, payment, verifica
│   ├── middleware/         # authenticate, authorizeRole, verifyPermission, scopeEstate, ra
│   ├── models/             # User, Estate, Property, UserProperty, Dependant, Invoice, Paym
│   ├── routes/             # auth.routes, users.routes, estates.routes, codes.routes, invo
│   ├── services/           # codeGenerator, paymentGateway, storageService, notificationSer
│   ├── validators/         # zodSchemas for all incoming payloads
│   ├── integrations/
│   │   └── onboarding/     # externalOnboardingSync.js (Idempotent engine)
│   ├── utils/              # crypto, logger, apiError
│   └── server.js
├── shared/
│   ├── constants/         # roles.js, statuses.js, validationMethods.js
│   ├── permissions/       # permissionMatrix.js
│   └── schemas/           # shared Zod validation schemas
├── .env.example
├── .gitignore
└── README.md

```

8. Immediate Next Steps for Implementation

- Scaffold Repository Structure:** Build the initial folder structure, package manifests (`client` , `server`), and `.env.example` .
- Setup Base Data Models & Seed Data:** Implement `User` , `Estate` , `Property` , and `UserProperty` schemas along with the 7-layer authorization middleware.

3. **Build External Onboarding Integration API:** Implement the idempotent push/pull sync engine to ingest residents seamlessly.
4. **Develop the Gate Verification Station & Access Code Engine:** Build the resident code generation portal and the guard camera PWA interface.